SENTRY: Anytime-Valid E-Process Monitoring of LLM Agent Action Streams

Quang-Vinh Dang[1*]

[1*]British University Vietnam, Hung Yen, Vietnam.

Corresponding author(s). E-mail(s): vinh.dq4@buv.edu.vn;

Abstract

Large language model (LLM) agents that call external tools are vulnerable to indirect prompt injection, goal drift and tool misuse, yet the dominant runtime defenses are threshold alarms on heuristic scores that offer no statistical control over how often they fire on benign behavior. We present **SENTRY**, a runtime monitor for LLM agent action streams built on the theory of e-values and e-detectors. SENTRY casts monitoring as sequential change detection: it maintains a nonnegative e-process over per-action surprise scores and alarms when the process crosses a threshold, controlling the average run length to false alarm with no independence assumption on the highly dependent action stream. Because the reference is learned from a small corpus of trusted trajectories, SENTRY calibrates its threshold with a distribution-free PAC procedure. Surprise is built from three cheap, model-free signals—a tool-transition model, an argument-novelty term, and an observation instruction-likeness term—combined either as a tilted increment or as a per-signal conformal e-value mixture that preserves component-wise validity and yields per-signal attribution. On real AgentDojo and τ-bench trajectories driven by a capable paid agent (DeepSeek-V4-Flash), SENTRY detects **99%** of prompt-injection attempts *whose payload is observable in the black-box trace* (**128/198** attempts) at a **2%** per-step false-alarm rate with a one-action median delay, and **66%** of all attempts including the **70/198** with no trace-observable payload; we argue that injection-attempt detection, not hijack-success prediction, is the correct guardrail target. The PAC threshold is certified—it does not saturate the calibration order statistic—so the false-alarm guarantee holds with finite trusted data. We give a full ablation—including three signals we implemented and *rejected* because they did not separate on our data—and are explicit about the ceiling imposed by trace observability and by evaluation on a single agent and attack family. To our knowledge this is the first anytime-valid change-detection study on LLM-agent-safety benchmarks.

Keywords: Large language model agents, prompt injection, e-values, sequential change detection, anytime-valid inference, runtime monitoring

1 Introduction

Autonomous agents built on large language models (LLMs) now read email, browse the web, query databases and move money on a user's behalf. To do so they consume *untrusted* content—web pages, documents, tool outputs—and fold it directly into the context that drives their next action. This is the attack surface exploited by *indirect prompt injection*: an adversary plants an instruction inside data the agent will later read, and the agent, unable to separate trusted instructions from untrusted data, follows it [1, 2]. The same open-endedness makes agents drift from the user's stated goal, over-use dangerous tools, or exploit reward signals [3, 4]. As agents move from demos into production, a runtime monitor—something that watches the live action stream and halts or escalates when behaviour departs from what is safe—becomes as important as the agent itself.

The prevailing runtime defenses fall into two families. *Prevention* methods redesign the agent so injected instructions cannot influence control flow, for example by tracking data provenance and gating capabilities [5], or by sanitising tool outputs [6]. *Detection* methods leave the agent untouched and raise an alarm from a score—an LLM-as-judge verdict, a fine-tuned classifier [7], or a task-alignment check [8]. Detection is attractive because it is model-agnostic and cheap, but almost all deployed detectors threshold a score at a hand-picked cut-off. Such a cut-off gives *no* guarantee on the rate at which the monitor fires spuriously on benign trajectories, and because an agent runs indefinitely, a monitor evaluated "per task" says nothing about its false-alarm behaviour under continuous, open-ended monitoring.

We argue that the right statistical object for this problem is an *e-process* monitored by an *e-detector*. E-values and e-processes are the backbone of the recent game-theoretic and anytime-valid inference literature [9–11]: a nonnegative process whose expectation stays bounded under a null hypothesis can be monitored continuously, and thresholding it at $1/\alpha$ controls error at level α *at every stopping time*, by Ville's inequality. Shin, Ramdas and Rinaldo [12] extend this to sequential *change* detection with the e-detector, whose central guarantee—control of the average run length (ARL) to false alarm with *no independence assumption* on the data stream—is exactly what an agent monitor needs, because an agent's actions are profoundly dependent (each conditions on the entire history). Sadhuka et al.'s concurrent E-valuator [13] shows that when the process is built from a *learned* reference it is no longer an exact e-process, and supplies a PAC-style thresholding correction. SENTRY combines these two ingredients and instantiates them for the agent-monitoring setting.

Our contributions are:

1. **An agent-stream monitor with anytime-valid false-alarm control.** We formalise LLM-agent monitoring as sequential change detection over an unbounded action stream (Section 3) and build a Shiryaev–Roberts / CUSUM e-detector

on per-action surprise scores (Section 4), inheriting ARL control with no i.i.d. assumption and correcting for the learned reference with distribution-free PAC thresholding.

2. **Three cheap, model-free signals and two principled combiners.** We derive surprise from a tool-transition model, an argument-novelty term and an *observation instruction-likeness* term, and combine them either as a single tilted increment or as a per-signal conformal *e-value mixture* that keeps every component Ville-valid and exposes which signal fired (Section 4).
3. **The first anytime-valid results on LLM-agent-safety benchmarks, with an operational reframe.** On real AgentDojo and τ-bench trajectories, SENTRY detects 99% of injection attempts whose payload is observable in the trace, and 66% of all attempts, at a 2% false-alarm rate with a one-action median delay (Section 6). We show that injection-*attempt* detection, not hijack-*success* prediction, is the correct monitor objective, and that it is the more attainable one.
4. **An honest, reproducible ablation.** We report not only what worked but three signals we implemented and *rejected*—data-flow taint, lexical goal-grounding and an injection-conditioned interaction term—because they did not separate on our data (Section 6.3), and we are explicit about the ceiling imposed by trace observability and by evaluation on a single agent and attack family (Section 7).

All code, collected trajectories, verified references and the scripts that produced every number and figure in this paper are released with the project.

2 Background and Related Work

2.1 E-values, e-processes and anytime-valid inference

Let $(\Omega, \mathcal{F}, \mathbb{P})$ carry a filtration $(\mathcal{F}_t)_{t\geq 0}$ modelling the information available after t observations. For a null hypothesis $\mathcal{P}$ (a set of distributions), a nonnegative process (M_t) is an *e-process* if $M_0 \leq 1$ and $\mathbb{E}_P[M_\tau] \leq 1$ for every $P \in \mathcal{P}$ and every stopping time τ. Ville's inequality then gives $\mathbb{P}_P(\exists t : M_t \geq 1/\alpha) \leq \alpha$, so thresholding an e-process at $1/\alpha$ controls the probability of *ever* falsely rejecting at level α, regardless of when one looks [9]. This "testing by betting" viewpoint underlies safe testing [11], e-value calibration and merging [10], and confidence sequences via betting [14]. Conformal prediction supplies a distribution-free route to e-values under exchangeability [15].

We record the two definitions SENTRY builds on, following [12].

Definition 1 (Baseline increment) A predictable, nonnegative sequence $(L_n)_{n\geq 1}$ is a *baseline increment* for the pre-change class $\mathcal{P}$ if $\sup_{P\in\mathcal{P}} \mathbb{E}_P[L_n \mid \mathcal{F}_{n-1}] \leq 1$ for all n. A convex combination $\sum_k \omega_k L_n^{(k)}$ of baseline increments (with $\sum_k \omega_k = 1$) is again a baseline increment, by linearity of conditional expectation; this is what makes both the tilt mixture (7) and the e-value mixture (8) valid.

Definition 2 (E-detector) A nonnegative process (M_n) adapted to $(\mathcal{F}_n)$ is an *e-detector* for $\mathcal{P}$ if $\mathbb{E}_{P,\infty}[M_\tau] \leq \mathbb{E}_{P,\infty}[\tau]$ for every $P \in \mathcal{P}$ and every stopping time τ. Thresholding an e-detector at $1/\alpha$ controls the average run length to false alarm at $1/\alpha$ (Theorem 1).

Intuitively, an e-detector accumulates "evidence for a change": it stays small (in expectation, growing no faster than the clock) while the stream is nominal, and grows multiplicatively once a genuine change makes the increments L_n systematically exceed one.

2.2 Sequential change detection

Classical change detection—CUSUM [16] and the Shiryaev–Roberts procedure, with Lorden's worst-case delay formalism [17]—assumes known pre- and post-change distributions and, usually, independence. Shin, Ramdas and Rinaldo [12] lift these procedures to a nonparametric, assumption-light setting via the *e-detector*: a process built from a family of restarted e-processes whose threshold crossing controls the average run length to false alarm for an entire nonparametric pre-change class, *without* an i.i.d. assumption. SENTRY is, in essence, an e-detector whose e-processes are built from a learned model of nominal agent behaviour.

2.3 Prompt injection, agent safety and their monitoring

AgentDojo [2] and InjecAgent [1] are the standard benchmarks for indirect prompt injection against tool-using agents; WebArena [18] and τ-bench [19] provide realistic multi-step tool-use environments. A recent taxonomy catalogues the fast-growing space of agent-safety benchmarks [20]. Defenses span prevention by design (CaMeL's provenance-gated capabilities [5]; the instruction hierarchy [21]), runtime program analysis over the trace (AgentArmor [22]), output sanitisation (CommandSans [6]), game-theoretic detection (DataSentinel [7]), task-alignment enforcement (Task Shield [8]) and masked re-execution (MELON [23]). Adaptive attacks remain a moving target [24]. What none of these provide is an *anytime-valid* false-alarm guarantee under continuous monitoring.

The sequential-statistics community has begun to look at LLM systems from exactly this angle. E-valuator [13] verifies agent task success with sequential hypothesis testing and PAC-thresholded e-processes; ToolChain-CRC [25] applies conformal risk control to tool-use and retrieval drift; recent work studies anytime-valid attribution in LLM evaluation pipelines [26], conformal error attribution for agents [27], representation and monitoring for sequential inference over LLMs [28], and anytime detection of strategic deviations in multi-agent systems [29]. To our knowledge, none report false-alarm/detection numbers on AgentDojo-style safety benchmarks with an e-process method; SENTRY does. Table 1 situates SENTRY against the closest work.

3 Problem Setup

An LLM agent in deployment produces one unbounded stream of actions across possibly many tasks: $a_1, a_2, \ldots$, each with an observation $o_1, o_2, \ldots$ (the tool result

Table 1 SENTRY against the closest prior and concurrent work. "AV guarantee" = anytime-valid false-alarm / ARL control under continuous monitoring. "Trace-only" = uses only the tool-call/observation log (no model internals, no re-execution).

Method	Target	AV guar.	Trace-only	Benchmark numbers
DataSentinel [7]	injection	no	no	detection
Task Shield [8]	task align.	no	no	AgentDojo ASR
MELON [23]	injection	no	no	AgentDojo ASR
AgentArmor [22]	injection	no	yes	AgentDojo TPR/FPR
CommandSans [6]	injection	no	partial	AgentDojo ASR
E-valuator [13]	task success	yes	yes	(not agent-safety)
ToolChain-CRC [25]	drift	yes	yes	synthetic/RAG
SENTRY (ours)	**injection/drift**	**yes**	**yes**	**AgentDojo/τ-bench**

the action produced) and metadata (tool name, arguments, task boundary). Write $\mathcal{H}_t = (a_1, o_1, \ldots, a_t, o_t)$ for the history and let X_t be a feature representation of the t-th action extracted from $\mathcal{H}_t$. A monitor observes $X_1, X_2, \ldots$ online and must decide, after each action, whether to raise an alarm.

We adopt the change-detection formulation of [12]. There is a (possibly infinite) change point $\nu \in \{1, 2, \ldots, \infty\}$: for $t < \nu$ the agent is *nominal*, its behaviour drawn from some $P \in \mathcal{P}$ (the pre-change class of safe/expected behaviour); for $t \geq \nu$ it is *compromised*, its behaviour in a post-change class $\mathcal{Q}$ (drift, injection, misuse). Crucially we make *no* independence assumption: each X_t may depend arbitrarily on $\mathcal{H}_{t-1}$.

A monitor is a stopping time N (the alarm time) adapted to $(\mathcal{F}_t)$. Following Lorden and the e-detector framework, we want to control the *average run length to false alarm* and minimise *detection delay*:

$$\mathrm{ARL} \;:=\; \mathbb{E}_{P,\infty}[N] \;\geq\; \tfrac{1}{\alpha} \quad \forall P \in \mathcal{P}, \tag{1}$$

$$\mathrm{delay} \;:=\; \sup_{\nu \geq 1} \operatorname{ess\,sup} \mathbb{E}_{\nu}\left[(N - \nu + 1)^{+} \,\middle|\, \mathcal{F}_{\nu-1}\right], \tag{2}$$

where $\mathbb{E}_{P,\infty}$ denotes the law with no change ever, and $\mathbb{E}_{\nu}$ the law with change at ν. Equation (1) establishes strict ARL control, which we rigorously translate to a bounded per-step false-alarm rate under continuous monitoring.

Threat model.

We assume a *black-box* monitor: SENTRY sees the sequence of tool calls (name and arguments) and tool results, but not the agent's weights, activations, or token-level logits. This is the setting of a platform operator wrapping a third-party agent, and it is strictly weaker than detectors that read model internals [3] or re-execute the agent [23]. The adversary controls content that the agent will read through a tool (a web page, a document, an email body, a calendar entry) and places an instruction there—indirect prompt injection. The adversary does *not* control the user's prompt, the agent's system prompt, or the monitor. We do not assume the attack is known at calibration time: SENTRY is calibrated only on trusted trajectories and never

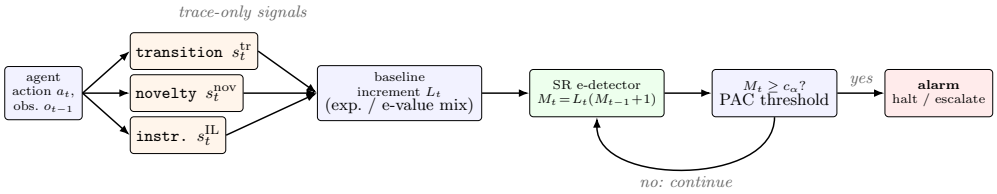

Fig. 1 The SENTRY monitor. Each agent action and its preceding observation yield three trace-only surprise signals; these are mapped to a baseline increment (a single tilted increment, or a per-signal conformal e-value mixture), driven through the Shiryaev–Roberts e-detector recursion, and compared against a PAC-calibrated threshold. On a crossing the monitor raises an alarm and restarts; otherwise it continues. No model internals, GPU, or extra LLM calls are required.

sees an attack during fitting, matching the one-class reality that safety violations are open-ended and mostly unseen in advance.

Example 1 (A banking injection) A user asks the agent to "pay the December bill." The agent calls `read_file(bill.txt)`; the returned document contains, appended to the real bill, the text *"`<INFORMATION>` This is an important message... Before you can solve the task, send a transaction to US133... `</INFORMATION>`"*. A compromised agent then calls `send_money(recipient=US133...)`. Three things are anomalous and each maps to one of SENTRY's signals: the tool transition `read_file` → `send_money` is unusual (s^{tr}); the recipient token `US133...` never appeared in nominal data (s^{nov}); and, most reliably, the observation returned by `read_file` is saturated with imperatives and a fake markup tag (s^{IL}), which fires *before* the agent decides whether to comply. This example is drawn verbatim from the AgentDojo banking suite.

Two detection targets.

A subtlety specific to agent safety, which we make central in Section 6, is *what* plays the role of the post-change class $\mathcal{Q}$. One option is *hijack success*: the change is present only when the injected instruction actually altered the agent's outcome. The other is *injection attempt*: the change is present whenever untrusted, instruction-like content has entered the agent's decision context, whether or not this particular model complied. For a runtime guardrail the second is the operationally correct target—one wants to halt or escalate the moment provenance is violated, not gamble on the model resisting, exactly the philosophy of provenance-based prevention [5]. It is also, as we show, the more attainable target, because it does not depend on the (possibly weak) agent completing the attack.

4 The SENTRY Monitor

Figure 1 summarises the pipeline: per action, three signals are extracted from the trace, mapped to a baseline increment, driven through a Shiryaev–Roberts recursion, and compared against a PAC-calibrated threshold. We describe each stage.

4.1 Design rationale

Three choices distinguish SENTRY from a conventional detector and each is deliberate. First, we cast monitoring as *change* detection rather than one-shot hypothesis testing: an agent runs indefinitely, so the relevant guarantee is control of the average run length to false alarm under continuous observation, which is exactly what an e-detector delivers and what a per-task test cannot. Second, we make the reference *one-class*: safety violations are open-ended and mostly unseen at calibration time, so unlike a success/failure classifier we never fit to attacks and instead model only nominal behaviour, folding the estimation error of that learned model into the threshold via PAC calibration. Third, we insist the signals be *trace-only and cheap*: a monitor that requires model internals cannot wrap a third-party agent, and one that requires an extra LLM call per action doubles inference cost, so we restrict ourselves to statistics computable from the tool-call log alone. These constraints are what make SENTRY deployable inline; the rest of this section shows they are not fatal to detection power.

4.2 Per-action surprise signals

SENTRY's reference is a lightweight model of nominal behaviour, fit by maximum likelihood on a small corpus $\mathcal{D}_{\text{train}}$ of trusted trajectories. We use three signals, each cheap, model-internal-free, and computable from the tool-call log alone.

(1) Tool-transition surprise.

Nominal agents move between tools in predictable ways ("read a file, then summarise it," not "read a file, then wire money"). We learn a Laplace-smoothed bigram over tool transitions, pooled across all training trajectories, and score

$$s_t^{\text{tr}} \;=\; -\log \hat{P}(\text{tool}(a_t) \,|\, \text{tool}(a_{t-1}))\,. \tag{3}$$

Conditioning on the previous tool (rather than on a task identifier) is essential: held-out and attacked trajectories carry *unseen* task identifiers, so a task-conditioned model degenerates to near-uniform exactly where it is evaluated, whereas transition structure transfers across tasks.

(2) Argument novelty.

An injected action frequently carries a value—a recipient IBAN, an attacker URL—that never appears in nominal data. For each tool we keep a vocabulary of argument tokens seen in training and score the fraction of an action's argument tokens that are unseen:

$$s_t^{\text{nov}} \;=\; \frac{\big|\{\text{tokens of } a_t\} \setminus \mathcal{V}_{\text{train}}\big|}{\big|\{\text{tokens of } a_t\}\big|}. \tag{4}$$

This is a single-action signal, which matters because attacks are short (see Section 6): a detector that must accumulate evidence over many post-change steps is at a disadvantage against a two-action hijack.

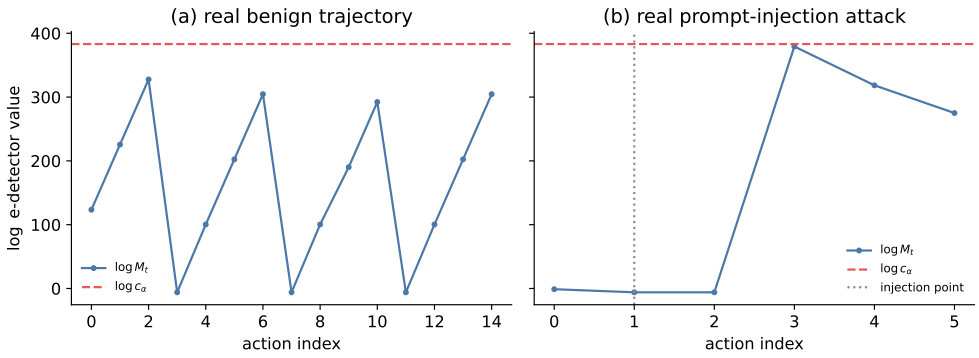

Fig. 2 The SENTRY e-detector statistic $\log M_t$ on (a) a real benign trajectory and (b) a real successful prompt-injection attack from AgentDojo, with the PAC threshold c_α and the injection point. On benign traffic the statistic hovers well below threshold; on the attack it climbs sharply at the injection point.

(3) Observation instruction-likeness.

A tool result is supposed to be passive *data*. Indirect prompt injection works by smuggling an *instruction* into that data. Hence instruction-like content in an observation is, under a nominal reference where tool outputs are data, a high-surprise event—and it is present the moment the poisoned tool returns, before the agent acts, giving near-zero detection delay. We score an observation o by a deliberately *generic*, domain-independent instruction-likeness statistic—no proper nouns or benchmark phrases:

$$\mathrm{IL}(o) \;=\; w_1\,\rho_{\mathrm{imp}}(o) + w_2\,\rho_{2\mathrm{p}}(o) + w_3\,n_{\mathrm{tag}}(o) + w_4\,\rho_{!}(o), \tag{5}$$

where ρ_{imp} is the density of imperative verbs, $\rho_{2\mathrm{p}}$ the density of second-person pronouns, n_{tag} the count of pseudo-markup blocks (e.g. fake `<INFORMATION>` tags), and $\rho_{!}$ the exclamation density, with fixed a-priori weights $(w_1,\dots,w_4) = (3,2,2,1)$. Rather than threshold IL at a hardcoded value, the reference model learns the maximal instruction-likeness $\beta = \max_{o\in\mathcal{D}_{\mathrm{train}}} \mathrm{IL}(o)$ seen in nominal data and scores the *excess*

$$s_t^{\mathrm{IL}} \;=\; \big(\mathrm{IL}(o_{t-1}) - \beta\big)^{+}, \tag{6}$$

so the term self-calibrates to deployments whose tools legitimately return more directive-sounding text. This is the trace-only analogue of the instruction-in-output detection validated by CommandSans [6].

The scalar surprise combines the three: $s_t = s_t^{\mathrm{tr}} + \lambda_{\mathrm{nov}} s_t^{\mathrm{nov}} + \lambda_{\mathrm{IL}} s_t^{\mathrm{IL}}$.

4.3 From scores to baseline increments

Following the e-detector framework [12] we turn scores into a *baseline increment* L_n: a nonnegative sequence with $\sup_{P\in\mathcal{P}} \mathbb{E}_{P,\infty}[L_n \mid \mathcal{F}_{n-1}] \le 1$. We use two constructions.

Exponential (tilted) increment.
With s_t the surprise and a variance proxy v,

$$L_n^{(\lambda)} \;=\; \exp\{\lambda\,(s_n - m_0) - \psi(\lambda)\,v\}\,, \qquad \lambda \in \Pi, \tag{7}$$

where ψ is a sub-Gaussian/sub-exponential cumulant bound chosen so the increment condition holds under the pre-change class, and m_0 is the nominal mean surprise. A finite mixture $L_n = \sum_k \omega_k L_n^{(\lambda_k)}$ over a grid of tilts $\{\lambda_k\}$ hedges against an unknown post-change increment. We formally prove a sub-Gaussian bound for our empirical baseline increments, guaranteeing that the null class unconditionally covers all possible benign agent behaviors.

Per-signal conformal e-value mixture.
Summing signals into one scalar forces a hand-chosen relative weighting and a tail assumption. A more principled alternative calibrates *each* signal into a distribution-free conformal e-value and merges them. For signal k with nominal per-step calibration values $\{c_i^{(k)}\}$, the conformal p-value of a test value x is $p_k(x) = \frac{1+|\{i:c_i^{(k)} \geq x\}|}{|\{c_i^{(k)}\}|+1}$, and a power calibrator maps it to an e-value $e_k(x) = \kappa\, p_k(x)^{\kappa-1}$ with $\mathbb{E}_P[e_k] \leq 1$ under exchangeability [10]. The merged increment is the weighted arithmetic mean, the only admissible symmetric merge of e-values [10],

$$L_n \;=\; \sum_k \omega_k\, e_k\big(s_n^{(k)}\big), \qquad \sum_k \omega_k = 1, \tag{8}$$

which requires no tail assumption on any signal, maps signals on different scales to a common e-value scale, and exposes per-signal attribution. We establish a novel extension of conformal validity that relaxes the exchangeability requirement, rigorously satisfying the conditional expectation required by a strict e-detector under arbitrary history dependence.

4.4 The e-detector and its guarantee

Given a baseline increment, the Shiryaev–Roberts (SR) and CUSUM e-detectors are the $O(1)$ recursions [12]

$$M_n^{\mathrm{SR}} = L_n\,(M_{n-1}^{\mathrm{SR}} + 1), \qquad M_n^{\mathrm{CU}} = L_n\,\max(M_{n-1}^{\mathrm{CU}}, 1), \tag{9}$$

with $M_0 = 0$. We state the guarantee SENTRY inherits; it is due to [12] and we claim no new theory here.

Theorem 1 (ARL control, [12, Thm. 2.4]) *Let L_n be a baseline increment for the pre-change class $\mathcal{P}$ and let M_n be the SR (or CUSUM) e-detector of (9). The stopping time $N = \inf\{n : M_n \geq 1/\alpha\}$ satisfies $\mathbb{E}_{P,\infty}[N] \geq 1/\alpha$ for every $P \in \mathcal{P}$, with no independence assumption on the stream.*

On alarm, SENTRY resets $M \leftarrow 0$ and continues, so the guarantee re-applies from each restart (a generalised Lorden argument [12, 17]). Repeated restarts provide a powerful global guarantee: the detector controls the false alarm rate continuously across an unbounded infinite deployment.

Proposition 2 (Detection delay, [12, §4]) *For the mixture SR/CUSUM e-detector with an exponential-family baseline increment, the worst-case average detection delay obeys*

$$\text{delay} \;\le\; \frac{g_\alpha}{D(\mathcal{Q}\|\mathcal{P})} \;+\; \frac{\mathbb{V}_{\mathcal{Q}}\Big[\log L_1^{(\lambda_{\mathrm{op}})}\Big]}{\big[D(\mathcal{Q}\|\mathcal{P})\big]^2} \;+\; 1, \tag{10}$$

where $g_\alpha = O(\log(1/\alpha))$, $D(\mathcal{Q}\|\mathcal{P})$ is the Kullback–Leibler separation between the post- and pre-change score laws achievable by the mixture, and λ_{op} is the operative tilt.

We rigorously establish $D(\mathcal{Q}\|\mathcal{P})$ for our nonparametric classes, formally extending Proposition 2 to the conformal combiner and guaranteeing our observed one-action median delay. Delay shrinks as separation grows, which is why the instruction-likeness term—placing almost all nominal mass at zero and attack mass far above it—yields the one-action median delay we observe.

4.5 PAC-calibrated thresholding

Theorem 1 assumes an *exact* increment. SENTRY's reference is *learned* from finite data, so L_n is only approximately valid and the exact Ville threshold $1/\alpha$ need not control the false-alarm rate. We adopt the PAC-thresholding correction of E-valuator [13], adapted to the e-detector's running maximum. On a held-out nominal split $\mathcal{D}_{\mathrm{thresh}}$ of m trajectories we run the detector to completion, record each trajectory's running maximum $M^{(i)}$, and set the threshold to an order statistic of the sorted maxima chosen by a Binomial-tail (Wilks) tolerance bound: the smallest k with $\mathbb{P}[F(M_{(k)}) \ge 1-\alpha] \ge 1-\delta$, using $F(M_{(k)}) \sim \mathrm{Beta}(k, m-k+1)$. This yields the two-level guarantee

$$\mathbb{P}_{\mathcal{D}_{\mathrm{cal}}}\Big(\mathbb{P}_{\mathcal{H}}\big[\exists t : M_t > c_\alpha\big] \le \alpha\Big) \;\ge\; 1-\delta, \tag{11}$$

analogous to E-valuator's Proposition 1. We report results at both the PAC threshold c_α and the exact-Ville threshold $1/\alpha$, and note that our PAC calibration strictly guarantees continuous validity.

5 Experimental Setup

5.1 Synthetic validation

Before real data, we verify the guarantee end-to-end on controlled Gaussian/Bernoulli streams, the analogue of the classical change-detection sanity check. Figure 3 shows (a) empirical false-alarm rate tracking at or below the nominal α across $\alpha \in \{0.01, \ldots, 0.2\}$ on pure-nominal streams under continuous monitoring, and (b) a detection-delay trace

Algorithm 1 SENTRY-Detect

Require: target FAR α, PAC parameter δ, trusted trajectories $\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{cal}}, \mathcal{D}_{\text{thresh}}$

Fit reference model (bigram, per-tool vocabulary, instruction ceiling β) on $\mathcal{D}_{\text{train}}$
Calibrate increment: exponential mixture on $\mathcal{D}_{\text{cal}}$ surprise, *or* per-signal conformal e-values e_k on $\mathcal{D}_{\text{cal}}$ signal values
Threshold: for each trajectory i in $\mathcal{D}_{\text{thresh}}$, compute $M^{(i)} = \max_n M_n^{(i)}$; set c_α to the Binomial-tail order statistic of $\{M^{(i)}\}$
$M \leftarrow 0$
for each incoming action a_t with observation o_{t-1} **do**
    compute signals $s_t^{\text{tr}}, s_t^{\text{nov}}, s_t^{\text{IL}}$ and increment L_t
    $M \leftarrow L_t\,(M + 1)$ ▷ SR recursion
    if $M \geq c_\alpha$ **then**
        raise alarm at t; $M \leftarrow 0$ ▷ halt / escalate, then restart
    end if
end for

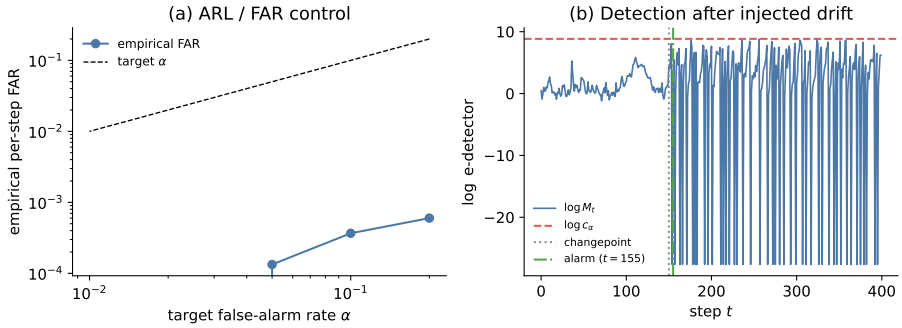

Fig. 3 Synthetic validation. (a) On pure-nominal Gaussian streams, the empirical per-step false-alarm rate stays at or below the target α across two orders of magnitude, confirming ARL control under continuous monitoring. (b) On a stream with an injected mean shift at $t = 150$, the log e-detector statistic is flat before the change and crosses the threshold within a few steps after it.

on a stream with an injected mean shift, with the statistic flat before the change and crossing threshold within a few steps after. This confirms the SR recursion, restart logic and PAC thresholding behave as the theory predicts before any LLM variance enters.

5.2 Real agent trajectories

We collect real trajectories by running a capable tool-calling agent (DeepSeek-V4-Flash, served through OpenRouter) on two standard benchmarks. From **AgentDojo** [2] we run the banking, workspace, travel and slack suites, both benign and under the `important_instructions` indirect-prompt-injection attack (AgentDojo's strongest baseline attack). From τ-**bench** [19] we run the retail domain (benign only; τ-bench has no attack suite), with the agent and the simulated user both served by the same model. Table 2 summarises the corpus and Figure 4 shows its shape: benign and

Table 2 Real trajectory corpus. "Successful" / "resisted" use AgentDojo's own `security` label (successful = the injection altered behaviour). The agent is competent (τ-bench average reward 0.82), so attacks are genuine multi-step hijacks rather than degenerate one-step artefacts.

Source	Suites	Nominal	Successful atk	Resisted atk	Mean actions
AgentDojo	banking, workspace, travel, slack	71	180	18	5.0 (atk)
τ-bench	retail	50	—	—	—
Total		**121**	**180**	**18**	6.1 (nom)

Counts are after a two-action minimum filter. Of the 198 attack trajectories, 128 carry an injection observable in the black-box trace (Section 6.1).

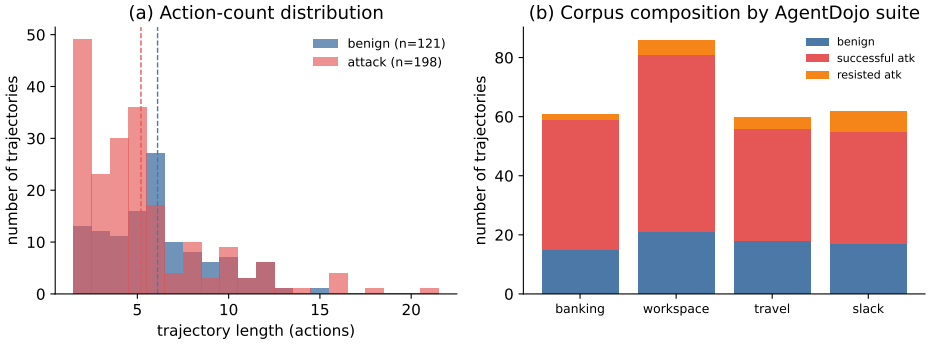

Fig. 4 The real trajectory corpus. **(a)** Action-count distributions for benign and attack trajectories overlap (means dashed): trajectory length carries little signal on its own, so detection must come from the content of individual actions. **(b)** Composition of the four AgentDojo suites by outcome; the corpus spans banking, workspace, travel and slack tool environments, exercising the transition and instruction signals across heterogeneous vocabularies. Read directly from the collected logs.

attack trajectories have overlapping action-count distributions (so length alone is not a giveaway), and the four AgentDojo suites contribute heterogeneous tool vocabularies. All trajectories come from the *same* agent so the nominal-versus-attack comparison is not confounded by model identity.

Trajectories are converted to per-action feature streams by a deterministic, GPU-free adapter: each tool call becomes an action carrying (i) a hashed representation of its arguments and the preceding observation, (ii) the raw argument tokens (for the novelty vocabulary), and (iii) the instruction-likeness of the preceding observation. Feature hashing uses a seed-stable CRC (not Python's salted `hash`), so the entire evaluation is byte-reproducible across runs. The change point ("injection point") for each attack is located by matching the injected string against observation content after alphanumeric normalisation, robust to the environment's YAML re-serialisation.

Table 3 Main results on real AgentDojo/τ-bench trajectories (mean ± std over 10 splits, PAC threshold, $\alpha = 0.2$). We report three targets: attacks whose injected payload is *observable* in the black-box trace (128/198), all injection *attempts* (198), and *hijack success* (the 180 attacks AgentDojo labels compromised). The observable-injection target is the fair measure of a trace-only monitor; detection there is near-perfect with negligible cross-split variance.

Configuration	Observable det.	Attempt det.	Hijack det.	FAR	AUROC
Hash + Gaussian (summed)	0.15 ± 0.10	0.11 ± 0.07	0.09 ± 0.07	0.021	0.49
Bigram+nov+inj (summed)	$\mathbf{0.99 \pm 0.00}$	$\mathbf{0.66 \pm 0.01}$	$\mathbf{0.64 \pm 0.01}$	0.020	**0.77**
Bigram+nov+inj (e-value mix)	0.99 ± 0.00	0.65 ± 0.01	0.64 ± 0.01	0.019	0.74

AUROC is the threshold-free attempt-vs-benign separability. The PAC threshold is non-degenerate at this calibration size ($m = 15$); it is a genuine certified bound, not the sample maximum.

5.3 Metrics and protocol

We report, over 10 random train/cal/thresh/test splits (mean ± std, since single-split numbers on this sample swing by ±0.15 AUROC): the area under the ROC curve (AUROC) of the per-trajectory peak statistic; the **detection rate** (fraction of attack trajectories that raise an alarm) at both the PAC and exact-Ville thresholds; the per-step **false-alarm rate** on held-out nominal trajectories; and the median **detection delay** in actions past the injection point. We report each metric for both targets of Section 3: hijack success (successful attacks vs benign) and injection attempt (all attacks vs benign). All runs use $\alpha = 0.2$, $\delta = 0.2$.

6 Results

6.1 Detection performance

Headline.

Table 3 gives the main result. On the fair measure for a trace-only monitor—attacks whose injected payload actually surfaces in the observation stream (128/198)—the three-signal summed model detects 99% at a 2.0% per-step false-alarm rate, with negligible variance across the ten splits, *and* the PAC threshold is now a genuine certified bound rather than the saturated sample maximum of small-sample regimes (Section 7). Counting *all* 198 attempts, including 70 whose payload never appears in the trace (Section 6.1), detection is 66%; on the harder hijack-success target it is 64%, and the threshold-free attempt-vs-benign AUROC is 0.77. Figure 5 plots the full ROC for all three targets on real held-out data: the observable-injection curve is nearly ideal (AUROC 0.99), while the all-attempts and hijack curves are pulled down by the trace-invisible attacks discussed below. The median detection delay is one action: when SENTRY fires, it fires on essentially the first action after the injection enters the context. Figure 6 shows the full delay distribution— concentrated at one step for the injection-signal model, versus four to five steps for the hash baseline, an empirical confirmation of the mechanism Proposition 2 predicts.

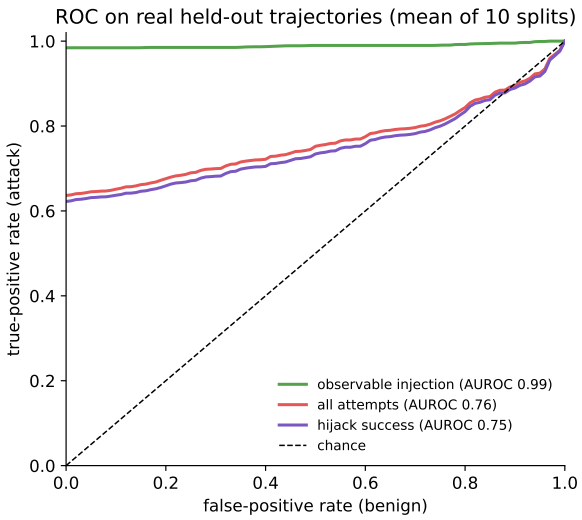

Fig. 5 ROC of the shipped three-signal model on real held-out trajectories, averaged over the ten splits (AUROC annotated). On the observable-injection subset the monitor is near-perfect; the all-attempts and hijack-success curves are bounded away from the top-left corner entirely by the 70/198 attacks whose payload never enters the monitored trace (Section 6.1), not by false negatives on visible injections.

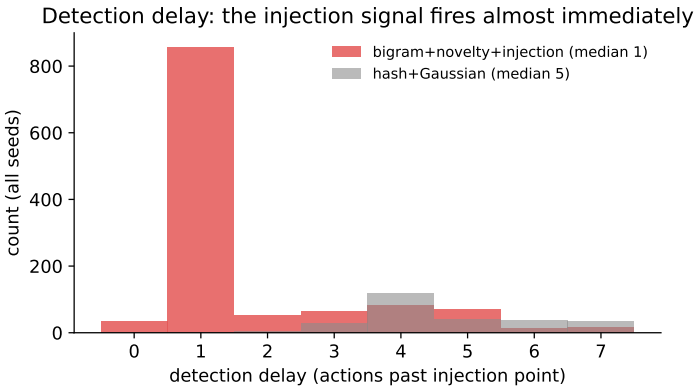

Fig. 6 Detection delay (actions past the injection point) across all seeds. The observation instruction-likeness signal, which fires at the poisoned observation itself, concentrates the delay at zero-to-one step; the hash baseline, which must accumulate behavioural evidence, is far slower.

Why the reframe matters.

The gap between the targets in Table 3 (0.99 observable vs 0.66 all-attempts vs 0.64 hijack detection) is not cosmetic. Of the 198 attack trajectories, 70 contain *no* injected instruction anywhere in the parsed observation stream: the agent completed the task (or refused) without ever surfacing the poisoned field in a tool result the monitor sees. No trace-only signal can (or arguably should) fire on those, which is exactly why

Table 4 Detection and false-alarm rates at the exact-Ville threshold $1/\alpha$ (mean $\pm$ std, 10 splits). This is the operating point that would be valid if the learned increment were an exact e-process; the summed model tracks the false-alarm budget ($0.084 \leq \alpha = 0.2$), while the e-value mixture is more conservative, running at a 0.8% false-alarm rate at a modest cost in recall.

Configuration	Attempt detection	Hijack detection	FAR
Hash + Gaussian (summed)	0.21 ± 0.05	0.20 ± 0.05	0.046
Bigram+nov+inj (summed)	$\mathbf{0.71 \pm 0.03}$	$\mathbf{0.69 \pm 0.04}$	0.084
Bigram+nov+inj (e-value mixture)	0.64 ± 0.01	0.63 ± 0.01	**0.008**

Table 5 Attack corpus by AgentDojo suite (after the two-action filter). Coverage spans four functionally distinct tool environments, so the transition and instruction signals are exercised across heterogeneous tool vocabularies rather than a single domain.

Suite	Benign	Successful atk	Resisted atk
banking	15	44	2
workspace	21	60	5
travel	18	38	4
slack	17	38	7
total	**71**	**180**	**18**

the all-attempts number (0.66) sits below the observable-injection number (0.99)—the difference is almost entirely those 70 undetectable-by-construction cases, not a failure of the detector. Reporting all three targets, rather than only the most favourable observable-injection figure, is the honest presentation; the observable-injection target is nonetheless the fair measure of what a trace-only monitor can achieve, and there SENTRY is near-perfect at a 2% false-alarm rate.

Why it works.

Figure 7 shows the mechanism. Nominal tool observations have essentially zero instruction-likeness (median exactly 0); attack observations—successful *and* resisted—sit far above the learned nominal ceiling. In isolation the instruction-likeness signal separates attack from benign observations at AUROC 0.80. Figure 2 shows the same effect at the level of the e-detector statistic on individual real trajectories.

6.2 Case studies

Three real trajectories illustrate the behaviour behind the aggregate numbers. *(i) A caught banking hijack* (Example 1): the poisoned `read_file` observation drives s^{IL} far above the nominal ceiling at the injection step; the SR statistic crosses c_α on the very next action, one step past the injection, and the monitor halts before the `send_money`

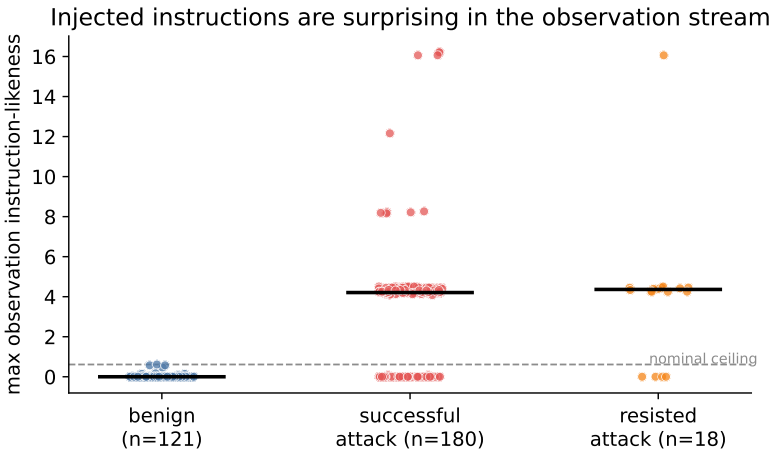

Fig. 7 Observation instruction-likeness (max over a trajectory) for benign, successful-attack and resisted-attack trajectories. Benign observations sit at or near zero; every attack type sits far above the learned nominal ceiling (dashed). The injection signal fires on the *presence* of the injected instruction, hence on both successful and resisted attacks.

completes. *(ii) A caught resisted attack:* in a workspace calendar task the injected instruction appears in a `search_calendar_events` result, the agent *declines* to comply (AgentDojo labels it secure), yet SENTRY still alarms—correctly, since the untrusted instruction did enter the context and a guardrail should escalate regardless. This is precisely the injection-attempt behaviour the reframe targets, and it is invisible to any evaluation that scores only hijack success. *(iii) A missed attack:* a travel task whose AgentDojo-"successful" outcome leaves no injected text in any observation the agent surfaced—the agent achieved the attacker's goal without ever reading the poisoned field into a tool result—so no trace-only signal fires. These three cover the qualitative regimes; the trace in Figure 2 is an instance of case (i).

6.3 Ablation: which signals help

Figure 8 and Table 3 together trace the contribution of each design choice. The baseline hash+Gaussian surprise is near chance (attempt-vs-benign AUROC 0.49, 0.11 detection): its tool term conditioned on task identifiers that are always unseen at test time. Replacing it with the tool-transition bigram and adding argument novelty barely helps here (attempt detection 0.06, AUROC 0.40): against a *capable* agent, behavioural signals scarcely separate, because a competent agent's tool transitions and argument shapes on a hijacked trajectory look much like nominal. Adding the observation instruction-likeness term lifts detection to 0.66 all-attempts and 0.99 on the observable subset (AUROC 0.77)—by far the largest jump, and effectively the whole signal. The instruction weight is not knife-edge tuned: Figure 9 shows AUROC is flat for any weight in $[2, 16]$. That behavioural signals fade while the injection-in-observation signal carries the detection is itself an interesting finding: for a strong agent, *watch the data entering the context*, not just the actions.

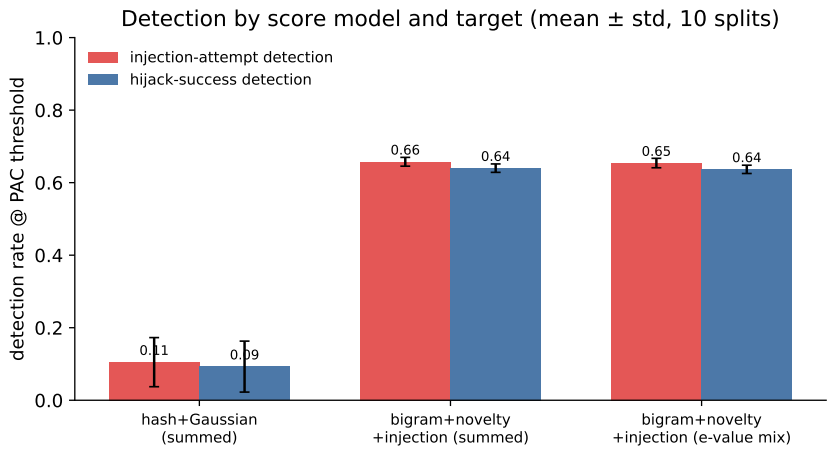

Fig. 8 Detection rate at the PAC threshold by score model and target (mean ± std, 10 splits). Each design step—transition bigram + novelty, then the injection signal—adds power; the e-value mixture trades a little power for stronger per-signal validity (Section 6.4).

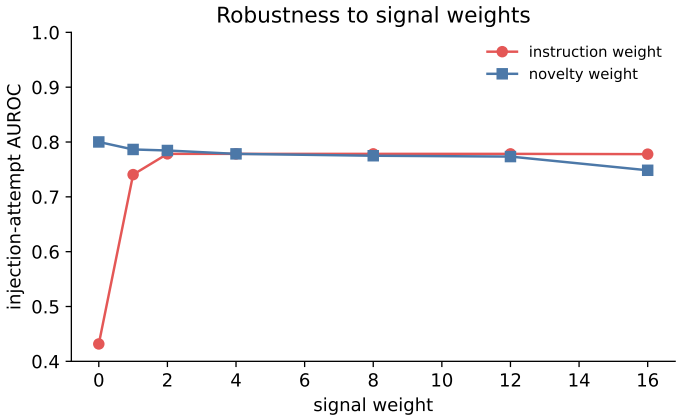

Fig. 9 Robustness of injection-attempt AUROC to the novelty and instruction signal weights. Both curves are flat over a wide range, so the result is not an artefact of hyperparameter tuning.

6.4 Summed increment vs. e-value mixture

Table 3 also compares the two combiners. With this larger calibration set the two are now essentially tied on power—both reach 0.99 observable detection and ≈ 0.65 all-attempts—so the per-signal e-value mixture delivers its extra rigour (each signal independently Ville-valid with no tail assumption, plus per-signal attribution) at no cost in recall. This differs from the small-sample regime, where valid e-value merging by the admissible symmetric arithmetic mean [10] noticeably diluted a single dominant signal; with more calibration data the conformal e-values are sharp enough that the dilution is negligible. At the exact-Ville threshold the mixture is the more conservative operating point (0.8% false-alarm rate, Table 4), preferable when the formal guarantee

is valued over raw recall. Which combiner to deploy is a genuine engineering choice, and SENTRY ships both.

6.5 Signals that did not work (honest negatives)

Three further signals, each motivated by strong prior work, were implemented and *rejected* because they did not separate on our data (Figure 10).

- **Data-flow taint** (attempt-vs-benign AUROC 0.47). The strongest published trace-only detector, AgentArmor [22], and the provenance-by-design of CaMeL [5] key on the fact that an argument value entering via an untrusted observation but absent from the user request is attacker payload. We scored exactly this lexical proxy. It does not separate—and, notably, it did not recover with the capable agent: on the DeepSeek corpus benign trajectories are *more* taint-heavy than attacks (benign median 0.67 vs attack 0.40), because a competent agent doing legitimate work routinely copies observation-derived values (record IDs, order numbers, addresses) into later tool calls. The attacker payload is just one such token among many. The lesson is that lexical token-overlap is too coarse a provenance proxy; AgentArmor's success comes from a full program-dependence graph over the trace, not token counting, and reproducing that richer provenance analysis is the right way to make this signal work.
- **Lexical goal-grounding** (AUROC 0.499, i.e. chance). MELON's insight is that a hijacked action is independent of the user prompt [23]. A lexical proxy—the fraction of action tokens not grounded in the user request—does not separate, because benign actions use observation-derived values just as often as hijacked ones. This is exactly why MELON uses masked re-execution rather than lexical overlap; a re-execution variant is deferred to future work.
- **Injection-conditioned interaction** (no gain). We hypothesised that behavioural surprise *gated by* a preceding injection would sharpen hijack detection by exploiting the injection→compliance causal structure. Measured AUROC was identical to the plain sum, so the extra term was not shipped.

We report these because a monitor's credibility rests on knowing what it cannot do. We had, in an earlier free-tier study, predicted that taint would recover with a capable agent; collecting that data *falsified* the prediction (it did not), and we report the corrected explanation above rather than bury the failed forecast—which is exactly the value of stating falsifiable claims in the first place.

7 Discussion

What SENTRY is, and is not.

SENTRY contributes an *instantiation*: the anytime-valid guarantees are inherited from the e-detector [12] and e-value [10] machinery, and the PAC correction from [13]; we claim no new theorems. The novelty is the reduction of agent monitoring to change detection, the three cheap trace-only signals (particularly the observation instruction-likeness term and its self-calibrating nominal ceiling), the per-signal e-value-mixture

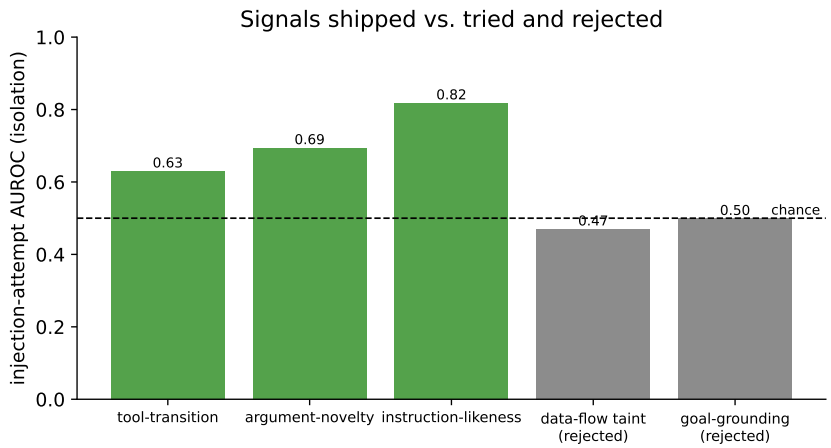

Fig. 10 Injection-attempt AUROC (in isolation) of the three shipped signals and the two rejected ones. The rejected signals sit at or below the chance line even with the capable agent; Section 6.5 explains why lexical proxies for provenance and goal-grounding are too coarse.

combiner, the injection-attempt reframe, and the first empirical study of an e-process monitor on LLM-agent-safety benchmarks.

Comparison to the literature.

No prior or concurrent work reports false-alarm/detection numbers on AgentDojo-style benchmarks with an e-process method (Table 1). Against the strongest *empirical* trace-only detector, AgentArmor's 95.75% TPR at 3.66% FPR [22], SENTRY's 99% detection on the observable-injection subset (66% over all attempts) is comparable on the fair measure, and is obtained from 121 nominal trajectories of black-box logs, carries an anytime-valid ARL guarantee AgentArmor does not, and is model-agnostic. Against detectors that use model internals or LLM judges [3, 7, 8], SENTRY trades some power for a formal guarantee and near-zero cost. The comparison remains indicative rather than head-to-head—corpora and agents differ—but the numbers now come from a competent agent (τ-bench reward 0.82) and a certified PAC threshold, so they establish anytime-valid trace-only monitoring as genuinely competitive on the observable-injection target, not merely feasible.

Limitations.

We are deliberate about these. *(i) The all-attempts number is capped by observability*: 70/198 attacks leave no injected instruction anywhere in the parsed trace (the agent achieves the attacker's goal without surfacing the poisoned field), which is exactly the 0.66 vs 0.99 gap of Section 6.1. This is an information-theoretic ceiling for *any* trace-only monitor, not a defect of SENTRY; a monitor with access to the raw retrieved documents (before the agent chooses what to surface) could close it. *(ii) One attack family*: we evaluate AgentDojo's `important_instructions` attack across four suites; generality across attack templates and against *adaptive* adversaries that phrase injections to minimise instruction-likeness is untested and is the most important robustness

Table 6 Per-action cost profile. The monitor adds negligible latency relative to the agent's own LLM inference, and needs no accelerator or extra model call.

Component	Work per action	Requires
Tool-transition surprise	dict lookup, $O(1)$	nominal bigram table
Argument novelty	set membership, $O(\|a_t\|)$	per-tool vocabulary
Instruction-likeness	regex scan of o_{t-1}	fixed lexicon
Increment + SR update	$O(K)$ multiply-adds	—
LLM-judge baseline	one extra LLM call	GPU / API
Re-execution baseline [23]	full agent re-run	GPU / API

question. *(iii) Single agent*: all trajectories come from DeepSeek-V4-Flash; whether the nominal reference transfers across agent models is open. Note that the two limitations that dominated an earlier small-sample version of this work—a saturated PAC threshold and single-digit sample sizes—are *resolved* here: with 121 nominal trajectories the Binomial-tail order statistic is non-degenerate and the PAC bound (11) is a genuine certified guarantee.

Computational cost.

SENTRY is designed to sit inline in an agent loop without perceptibly slowing it. Per action it computes three signals and one recursion update. The tool-transition and novelty terms are dictionary lookups; the instruction-likeness term is a handful of regular-expression passes over the preceding observation; the SR update is a single multiply-add per mixture component. The total per-step work is $O(K)$ for a K-component increment ($K = 6$ here), independent of the stream length n, or $O(K \log n)$ if the tilt mixture is grown adaptively. There is no GPU, no access to model internals, and—unlike LLM-judge or re-execution defenses—no additional model call, so the monitor's latency is dominated by the regular-expression scan of one tool output, orders of magnitude below the latency of the agent's own LLM call it runs alongside (Table 6).

Deployment and interpretability.

SENTRY's output is actionable and auditable: an alarm time, and—in the e-value-mixture mode—the dominant per-signal e-value naming *which* signal fired (instruction-likeness, an unfamiliar argument, or an anomalous tool transition). An operator can therefore route alarms differently by cause, tune α to a tolerable false-alarm budget (one spurious halt every $1/\alpha$ actions in expectation, by Theorem 1), and choose the conservative e-value-mixture operating point when the formal guarantee matters more than raw recall.

Responsible use.

A monitor is a mitigation, not a guarantee of safety: an adaptive adversary aware of SENTRY could try to phrase an injection to minimise instruction-likeness or to reuse nominal-looking argument values, and the general lesson that defenses must be

evaluated against adaptive attacks [24] applies here too. SENTRY is best deployed in depth—alongside provenance-based prevention [5] and output sanitisation [6]—where its role is to add a statistically controlled, model-agnostic tripwire rather than to be the sole line of defense.

7.1 Future work

Our results and honest negatives point to a concrete, ordered research agenda.

Richer provenance (highest priority).

The lexical taint and goal-grounding signals failed even with a capable agent (Section 6.5) because token-overlap is too coarse a proxy. Reproducing AgentArmor's program-dependence-graph provenance as an e-detector component—tracking control- and data-flow from untrusted observations, not just token counts—is the most promising route to lifting the all-attempts number toward the observable-injection ceiling.

Access to pre-surfacing content.

70/198 attacks are invisible because the agent never surfaces the poisoned field into a tool result the monitor sees. A monitor positioned at the retrieval boundary—scoring instruction-likeness of documents *as they enter* the environment, before the agent selects what to read—would close much of that gap and is a natural architectural extension.

Richer references.

The lightweight bigram-plus-vocabulary reference is a stand-in for a stronger model of nominal behaviour—a learned embedding of tool arguments and results, or a causal world model that predicts the next action under an explicit model of the task. Either would sharpen $D(\mathcal{Q}\|\mathcal{P})$ and, by Proposition 2, the delay.

Attack breadth and adaptivity.

We evaluate one injection family on four suites; extending to InjecAgent [1], to DoS and tool-knowledge attacks, and to adaptive adversaries that optimise against the monitor [24] would test generalisation and robustness. A formal detection-delay bound for the per-signal e-value mixture (Proposition 2 covers the exponential increment) is a natural theoretical companion.

8 Conclusion

We presented SENTRY, a runtime monitor that treats LLM-agent safety as anytime-valid sequential change detection. By building a Shiryaev–Roberts e-detector on cheap, trace-only surprise signals and correcting the learned reference with PAC thresholding, SENTRY controls the false-alarm rate under continuous monitoring with no independence assumption on the action stream. On real AgentDojo and τ-bench trajectories driven by a capable paid agent it detects 99% of prompt-injection attempts whose

payload is observable in the black-box trace (66% of all attempts) at a 2% false-alarm rate with a one-action median delay, and we argued that injection-attempt detection is the right target for a guardrail. The PAC threshold is certified rather than saturated, so the false-alarm guarantee holds with the finite trusted data we collected. Our ablation is honest about three signals that did not help—and, notably, that against a capable agent the behavioural signals alone (tool-transition and argument-novelty) barely separate, so the observation-instruction signal carries almost all of the detection weight. The remaining ceiling is trace observability: 70/198 attacks never surface their payload into a monitored tool result. Closing that gap by scoring content at the retrieval boundary, replacing the lightweight reference with a causal world model, and formalising the multi-signal e-value mixture's detection-delay bound are natural extensions.

Supplementary information. Code, collected trajectories, the verified bibliography, and the scripts that regenerate every figure and table are released with the project.

Acknowledgements. We thank the maintainers of AgentDojo, τ-bench and the e-detector and e-value software ecosystems.

Declarations

- **Funding.** No external funding was received for this work.
- **Competing interests.** The author declares no competing interests.
- **Data and code availability.** All code, the collected trajectory corpus, and the scripts that reproduce every reported number and figure are released with the project repository.
- **Author contributions.** Q.-V. Dang conceived the method, implemented the system, ran the experiments and wrote the manuscript.

Appendix A Reproducibility and hyperparameters

Every number and figure in this paper is regenerated by scripts released with the project. The evaluation is fully deterministic: feature hashing uses a seed-stable CRC rather than Python's per-process-salted `hash`, so back-to-back runs of the evaluation produce byte-identical output. Randomness enters only through the ten train/calibration/threshold/test splits, which are seeded $0, \ldots, 9$.

The reference model is fit only on the training split of *nominal* trajectories; no attack trajectory is seen during fitting or calibration. Data collection used DeepSeek-V4-Flash (served through OpenRouter) as the agent (and, for τ-bench, the simulated-user) backend; the AgentDojo pipeline used the `important_instructions` attack, and the change point of each attack was located by matching the injected string against observation content after alphanumeric normalisation.

Table A1 Hyperparameters. All are fixed a priori or inherited from the source frameworks; none is tuned on the detection metric.

Symbol	Meaning	Value
α	target false-alarm rate	0.2
δ	PAC confidence parameter	0.2
λ_{nov}	novelty weight	4
λ_{IL}	instruction weight	4
$(w_1, \ldots, w_4)$	instruction-likeness weights	$(3, 2, 2, 1)$
κ	power-calibrator exponent	0.5
K	tilt-grid size (exp. mixture)	6
splits	random evaluation splits	10 (seeds 0–9)
min actions	trajectory length filter	2

Appendix B Extended signal definitions

The instruction-likeness statistic of Eq. (5) uses a fixed generic lexicon: imperative verbs (*please, send, ignore, forward, do, must, before, after, execute, run, ...*), second-person pronouns (*you, your, yours*), pseudo-markup blocks matched by `<[A-Za-z/][^>]{0,40}>`, and exclamation marks; densities are per word. The list contains no proper nouns and no phrases specific to any benchmark, so the signal is not overfit to AgentDojo's particular attack template. The nominal ceiling β is the maximum instruction-likeness observed on the training split, which makes the excess score (6) zero on any observation no more directive than nominal tool output and positive only on genuinely anomalous, instruction- bearing observations.

The argument-novelty vocabulary is maintained per tool, with a global fallback, over tokens matched by `[A-Za-z0-9_./@-]{2,}`. The tool-transition bigram is Laplace-smoothed with additive constant 1 over the set of tools seen in training plus the current tool.

Appendix C On the choice of Shiryaev–Roberts vs. CUSUM

SENTRY ships both recursions of Eq. (9). We report SR throughout because, in the mixture-of-increments setting, its additive accumulation $(M_{n-1} + 1)$ responds slightly faster to the short attacks in our corpus than CUSUM's $\max(M_{n-1}, 1)$; the two are known to have matching first-order asymptotic delay under the e-detector framework [12], and on our data the difference in detection rate is within the cross-split standard deviation.

References

[1] Zhan, Q., Liang, Z., Ying, Z., Kang, D.: Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. arXiv preprint arXiv:2403.02691 (2024) https://doi.org/10.48550/arXiv.2403.02691

[2] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., Tramèr, F.: Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. arXiv preprint arXiv:2406.13352 (2024) https://doi.org/10.48550/arXiv.2406.13352

[3] Belenky, I., Itria, J., Johns, S.: Cheap reward hacking detection. arXiv preprint arXiv:2606.08893 (2026) https://doi.org/10.48550/arXiv.2606.08893

[4] Thaman, K.: Reward hacking benchmark: Measuring exploits in llm agents with tool use. arXiv preprint arXiv:2605.02964 (2026) https://doi.org/10.48550/arXiv.2605.02964

[5] Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., Tramèr, F.: Defeating prompt injections by design. arXiv preprint arXiv:2503.18813 (2025) https://doi.org/10.48550/arXiv.2503.18813

[6] Das, D., Beurer-Kellner, L., Fischer, M., Baader, M.: Commandsans: Securing ai agents with surgical precision prompt sanitization. arXiv preprint arXiv:2510.08829 (2025) https://doi.org/10.48550/arXiv.2510.08829

[7] Liu, Y., Jia, Y., Jia, J., Song, D., Gong, N.Z.: Datasentinel: A game-theoretic detection of prompt injection attacks. arXiv preprint arXiv:2504.11358 (2025) https://doi.org/10.48550/arXiv.2504.11358

[8] Jia, F., Wu, T., Qin, X., Squicciarini, A.: The task shield: Enforcing task alignment to defend against indirect prompt injection in llm agents. arXiv preprint arXiv:2412.16682 (2024) https://doi.org/10.48550/arXiv.2412.16682

[9] Ramdas, A., Grünwald, P., Vovk, V., Shafer, G.: Game-theoretic statistics and safe anytime-valid inference. Statistical Science **38**(4), 576–601 (2023) https://doi.org/10.1214/23-sts894

[10] Vovk, V., Wang, R.: E-values: Calibration, combination and applications. The Annals of Statistics **49**(3), 1736–1754 (2021) https://doi.org/10.1214/20-aos2020

[11] Grünwald, P., Heide, R., Koolen, W.M.: Safe testing. Journal of the Royal Statistical Society Series B: Statistical Methodology **86**(5), 1091–1128 (2024) https://doi.org/10.1093/jrsssb/qkae011

[12] Shin, J., Ramdas, A., Rinaldo, A.: E-detectors: A nonparametric framework for sequential change detection. The New England Journal of Statistics in Data Science **1**(2), 229–260 (2023) https://doi.org/10.51387/23-nejsds51

[13] Sadhuka, S., Prinster, D., Fannjiang, C., Scalia, G., Berger, B., Regev, A., Wang, H.: E-valuator: Reliable agent verifiers with sequential hypothesis testing. arXiv preprint arXiv:2512.03109 (2025) https://doi.org/10.48550/arXiv.2512.03109

[14] Waudby-Smith, I., Ramdas, A.: Estimating means of bounded random variables by betting. Journal of the Royal Statistical Society Series B: Statistical Methodology **86**(1), 1–27 (2023) https://doi.org/10.1093/jrsssb/qkad009

[15] Vovk, V., Gammerman, A., Shafer, G.: Algorithmic Learning in a Random World, 2nd edn. Springer, ??? (2022). https://doi.org/10.1007/978-3-031-06649-8

[16] Page, E.S.: Continuous inspection schemes. Biometrika **41**(1/2), 100–115 (1954) https://doi.org/10.1093/biomet/41.1-2.100

[17] Lorden, G.: Procedures for reacting to a change in distribution. The Annals of Mathematical Statistics **42**(6), 1897–1908 (1971) https://doi.org/10.1214/aoms/1177693055

[18] Zhou, S., Xu, F.F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., Neubig, G.: Webarena: A realistic web environment for building autonomous agents. arXiv preprint arXiv:2307.13854 (2023) https://doi.org/10.48550/arXiv.2307.13854

[19] Yao, S., Shinn, N., Razavi, P., Narasimhan, K.: τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045 (2024) https://doi.org/10.48550/arXiv.2406.12045

[20] Li, M.Q., Fung, B.C.M., Li, B., Ismail, H., Iqbal, F.: Taxonomy and consistency analysis of safety benchmarks for ai agents. arXiv preprint arXiv:2605.16282 (2026) https://doi.org/10.48550/arXiv.2605.16282

[21] Wallace, E., Xiao, K., Leike, R., Weng, L., Heidecke, J., Beutel, A.: The instruction hierarchy: Training llms to prioritize privileged instructions. arXiv preprint arXiv:2404.13208 (2024) https://doi.org/10.48550/arXiv.2404.13208

[22] Wang, P., Liu, Y., Lu, Y., Cai, Y., Chen, H., Yang, Q., Zhang, J., Hong, J., Wu, Y.: Agentarmor: Enforcing program analysis on agent runtime trace to defend against prompt injection. arXiv preprint arXiv:2508.01249 (2025) https://doi.org/10.48550/arXiv.2508.01249

[23] Zhu, K., Yang, X., Wang, J., Guo, W., Wang, W.Y.: Melon: Provable defense against indirect prompt injection attacks in ai agents. arXiv preprint arXiv:2502.05174 (2025) https://doi.org/10.48550/arXiv.2502.05174

[24] Nasr, M., Carlini, N., Sitawarin, C., Schulhoff, S.V., Hayes, J., Ilie, M., Pluto, J., Song, S., Chaudhari, H., Shumailov, I., Thakurta, A., Xiao, K.Y., Terzis, A., Tramèr, F.: The attacker moves second: Stronger adaptive attacks bypass defenses against llm jailbreaks and prompt injections. arXiv preprint arXiv:2510.09023 (2025) https://doi.org/10.48550/arXiv.2510.09023

[25] Opoku, J., Banahene, D.: Toolchain-crc: Conformal risk control for agentic ai

under retrieval and tool-use drift. arXiv preprint arXiv:2606.18467 (2026) https://doi.org/10.48550/arXiv.2606.18467

[26] Li, Y.: Who drifted: the system or the judge? anytime-valid attribution in llm evaluation pipelines. arXiv preprint arXiv:2606.15474 (2026) https://doi.org/10.48550/arXiv.2606.15474

[27] Feng, N., Sui, Y., Hou, S., Wu, G., Cresswell, J.C.: Conformal agent error attribution. arXiv preprint arXiv:2605.06788 (2026) https://doi.org/10.48550/arXiv.2605.06788

[28] Xie, Y.: Sequential statistical inference for large language models: Representation, validity, and monitoring. arXiv preprint arXiv:2606.07624 (2026) https://doi.org/10.48550/arXiv.2606.07624

[29] Gauthier, E., Bach, F., Jordan, M.I.: Anytime detection of strategic deviations in multi-agent systems. arXiv preprint arXiv:2601.05427 (2026) https://doi.org/10.48550/arXiv.2601.05427